Assignment 3

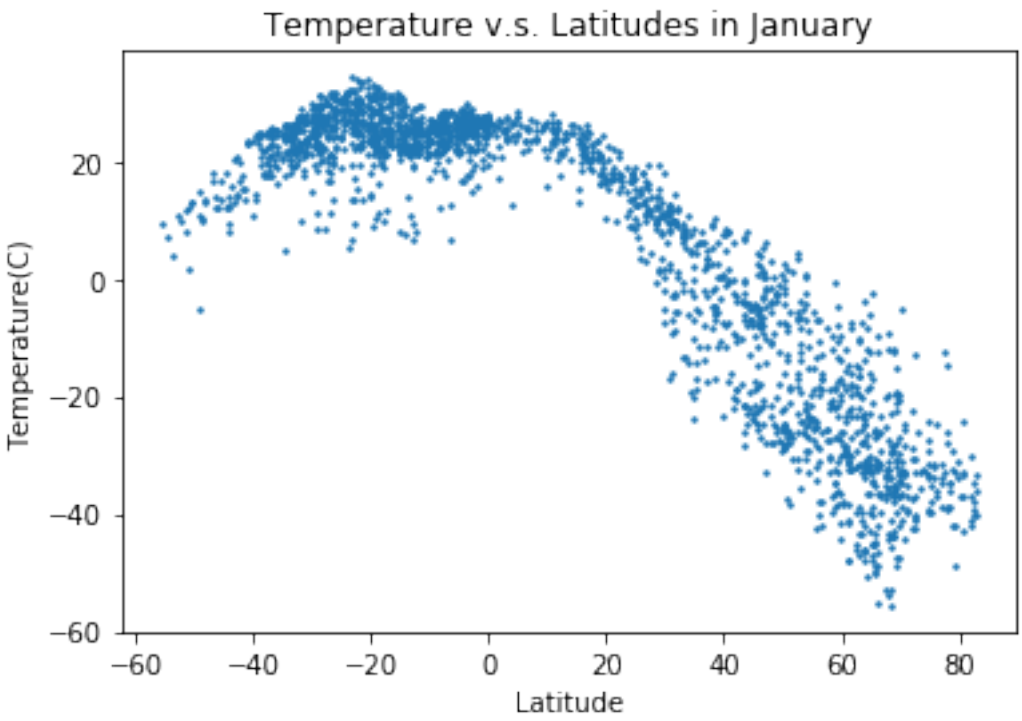
November 5, 2024

0.1 Problem 1

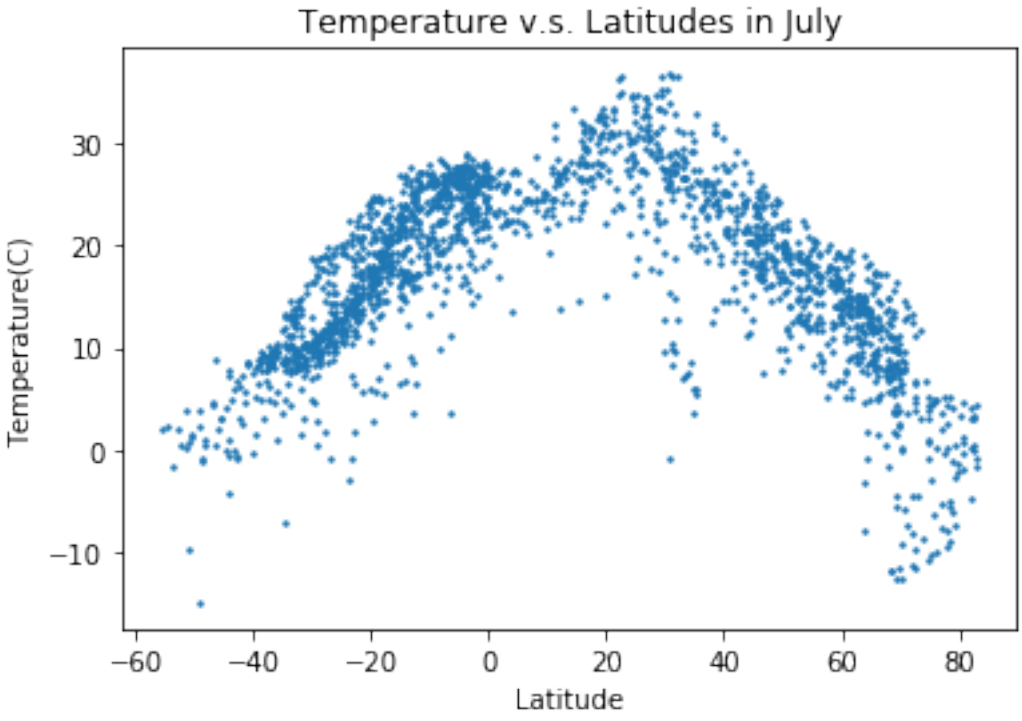
[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.formula.api as smf
```

0.1.1 1a)

[3]:
```
import pandas as pd
import matplotlib.pyplot as plt

df=pd.read_csv('shared/data/temperature.csv')

plt.scatter(df.latitude,df['1'],s=2)
plt.xlabel('Latitude')
plt.ylabel('Temperature(C)')
plt.title('Temperature v.s. Latitudes in January')
plt.show()
print('Correlation between temperature and latitude in Jaunary is',df['1'].
 ↪corr(df['latitude']))

plt.scatter(df.latitude,df['7'],s=2)
plt.xlabel('Latitude')
plt.ylabel('Temperature(C)')
plt.title('Temperature v.s. Latitudes in July')
plt.show()
print('Correlation between temperature and latitude in July is',df['7'].
 ↪corr(df['latitude']))
```



Correlation between temperature and latitude in Jaunary is -0.8808243838931636



```
Correlation between temperature and latitude in July is -0.09081110315246951
```

0.1.2 1b)

Traditionally, we put the independent variable on the x-axis and the dependent variable on the y-axis to study the effect of the independent variable on the dependent variable. Since we want to study the influence of latitude on temperature, it is natural that we put latitude on the x-axis and temperature on the y-axis.

0.1.3 1c)

As shown in the code, when flipping the order of the independent variable and dependent variable, the correlation still remains the same. The correlation does not depend on which variable is treated as X because correlation only shows how strongly two variables associate with each other. It does not have any causation connotation, which means correlation does not measure how strongly the independent variable affects the dependent variable. When causation is not established, the order of the independent variable and the dependent variable does not matter.

[5]:
```
df['latitude'].corr(df['1'])
```

[5]:
```
-0.8808243838931636
```

0.1.4 1d)

-Pattern of the graph in January

From the equator to -60 latitude, a major part of the data points are clustered around 20 Celsius. As the graph travels north of the equator, the data points are more scattered, and the temperature tends to decrease with the increase in latitude in the northern hemisphere.

-Explanation of the Pattern

The overall shape of the pattern could be explained by seasonality created across the southern hemisphere and the northern hemisphere. With the southern hemisphere still in the summer in January, the majority of the hemisphere would have higher temperatures caused by more sunlight. The northern hemisphere, on the other hand, is still in winter, and the temperature decreases with the increment in latitude. The equator gets the most direct sunlight no matter of season, which is why the temperature around latitude 0 is relatively high.

The southern hemisphere misses data from latitude -60 to latitude -80, which might cause the clustering data points to be around 20 Celsius. It is reasonable to assume that Antarctica is a lot colder even in the summer compared to the rest of the southern hemisphere. Another explanation for the potential clustered data points in the southern hemisphere is that there could be the temperature recording device can be denser on the map(more devices in an area) compared to the density of the recording device in the northern hemisphere.

-Magnitude of Correlation

According to the calculation, latitude and temperature display a strongly negative correlation, which means that when latitude increases the temperature decreases.

-Pattern of the graph in July

Compared to the January graph, the July graph is more shaped like a horseshoe, which means that the graph looks more balanced. With the highest temperature center around the equator, the temperature starts to decrease as the latitude gets further away from the equator.

-Explanation of the Pattern

One of the main reasons why the graph in July looks very different from that of January is that the northern hemisphere's ranges of temperature decrease in July compared to the ranges of temperature in January, while the southern hemisphere's ranges of temperature in July increase compared to the ranges of temperature in January. This characteristic could be attributed to the amount of sunlight each hemisphere is getting in different seasons, and could also be the missing range of data in the southern hemisphere from latitude -60 to -80.

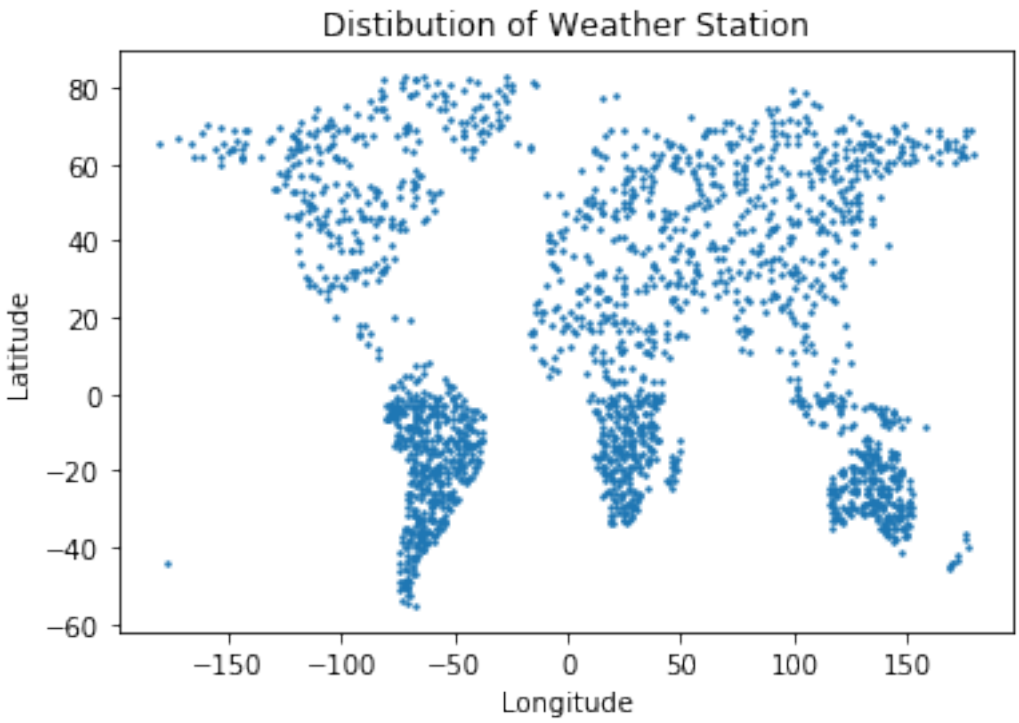
-Magnitude of Correlation

According to the calculation, latitude and temperature also have a negative correlation in July. The correlation is very weak due to the fact that the temperature starts to decrease as the latitude gets further away from the equator for both the southern hemisphere and the northern hemisphere, which demonstrates a weak negative correlation.

0.1.5 1e)

Based on the plot, the distribution of weather stations is not random with denser distribution in the southern hemisphere. The maximum latitude is 82.75 degrees, while the minimum latitude is -55.25 degrees. The difference in ranges between the northern and the southern hemispheres strengthens the correlation between temperature and hemisphere, especially for the graph in January. If the data set includes data from Antarctica, then the ranges of temperature would be a lot greater, which means that the graph will take on a horseshoe shape weakening the Pearson correlation(linear) between the two variables.

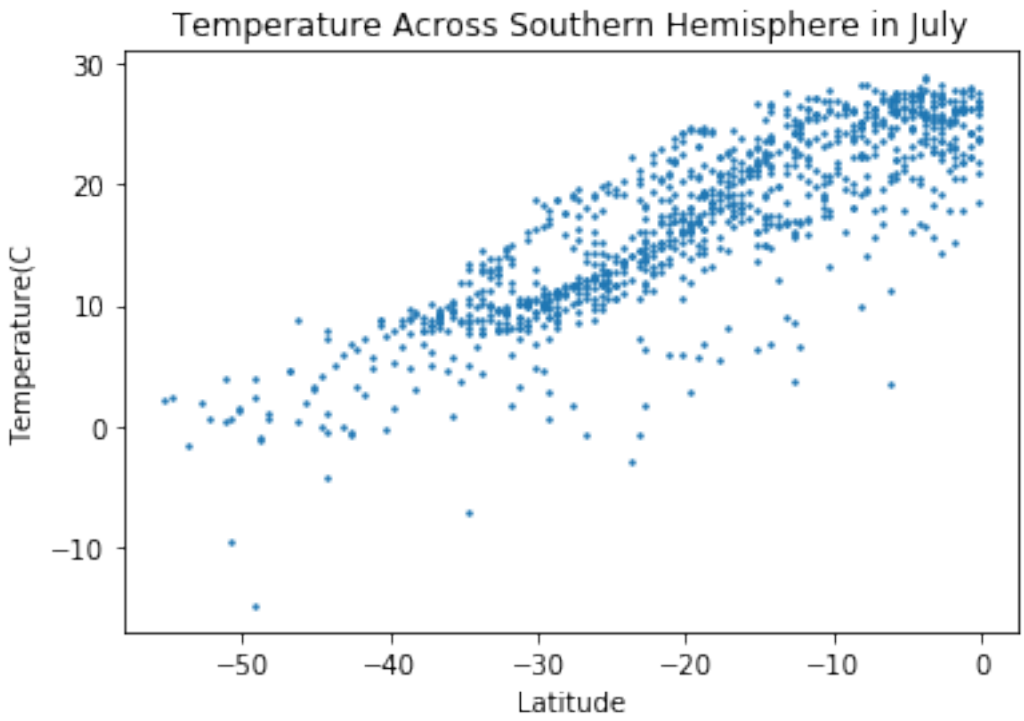
[11]:
```
import numpy as np

plt.scatter(df.longitude,df.latitude, s =2)
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.title('Distibution of Weather Station')
plt.show()

print('The maximum latitude is',np.max(df.latitude))
print('The minimum latitude is',np.min(df.latitude))
```

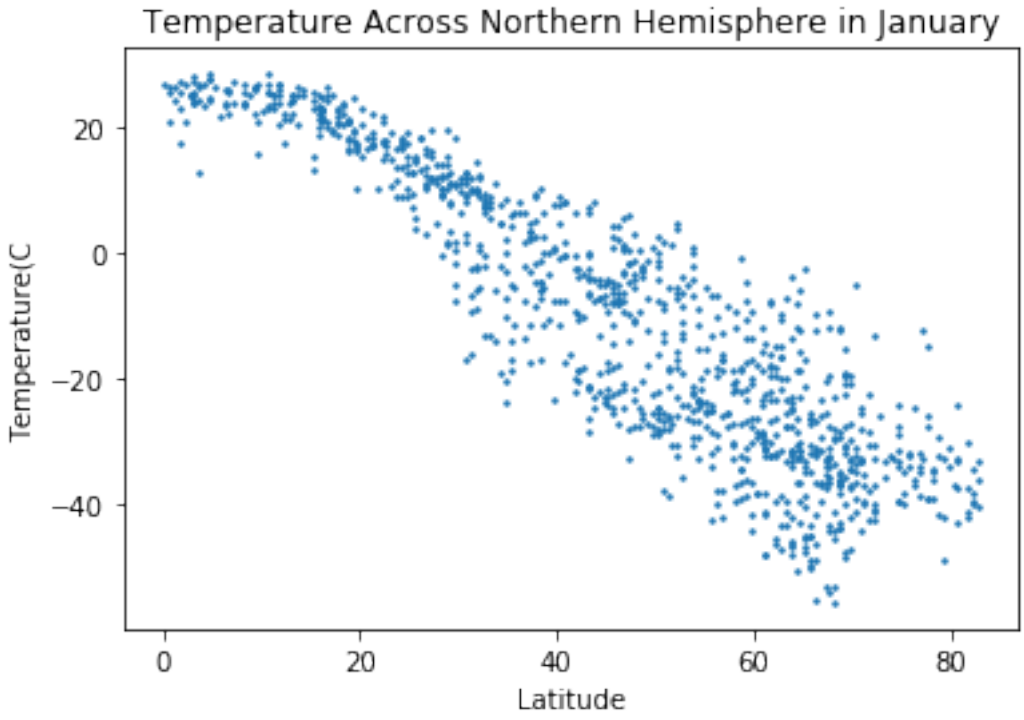



```
The maximum latitude is 82.75
The minimum latitude is -55.25
```

0.1.6 1f)

[15]:
```
df_south = df[df['latitude'] < 0]
plt.scatter(df_south['latitude'],df_south['7'],s=2)
plt.xlabel('Latitude')
plt.ylabel('Temperature(C')
plt.title('Temperature Across Southern Hemisphere in July')
plt.show()

df_north = df[df['latitude'] > 0]
plt.scatter(df_north['latitude'],df_north['1'],s=2)
plt.xlabel('Latitude')
plt.ylabel('Temperature(C')
plt.title('Temperature Across Northern Hemisphere in January')
plt.show()

print('The correlation between the northern hemispheres temperature and␣
 ↪latitude in January is',df_north['1'].corr(df_north.latitude))
```


Temperature Across Southern Hemisphere in July
Temperature(C
30
20
10
0
−10
−50
−40
−30
−20
−10
0
Latitude


Temperature Across Northern Hemisphere in January
Temperature(C
20
0
−20
−40
0
20
40
60
80
Latitude

```
The correlation between the northern hemispheres temperature and latitude in
January is -0.9044282825621138
```

0.1.7 1g)

-Notice

Stratified by hemisphere, the correlation between latitude and temperature is a lot stronger compared to that of the previous graphs. The highest temperature in both graphs is centered around the equator, and as the latitude moves away from the equator, the temperature decreases.

-Why?

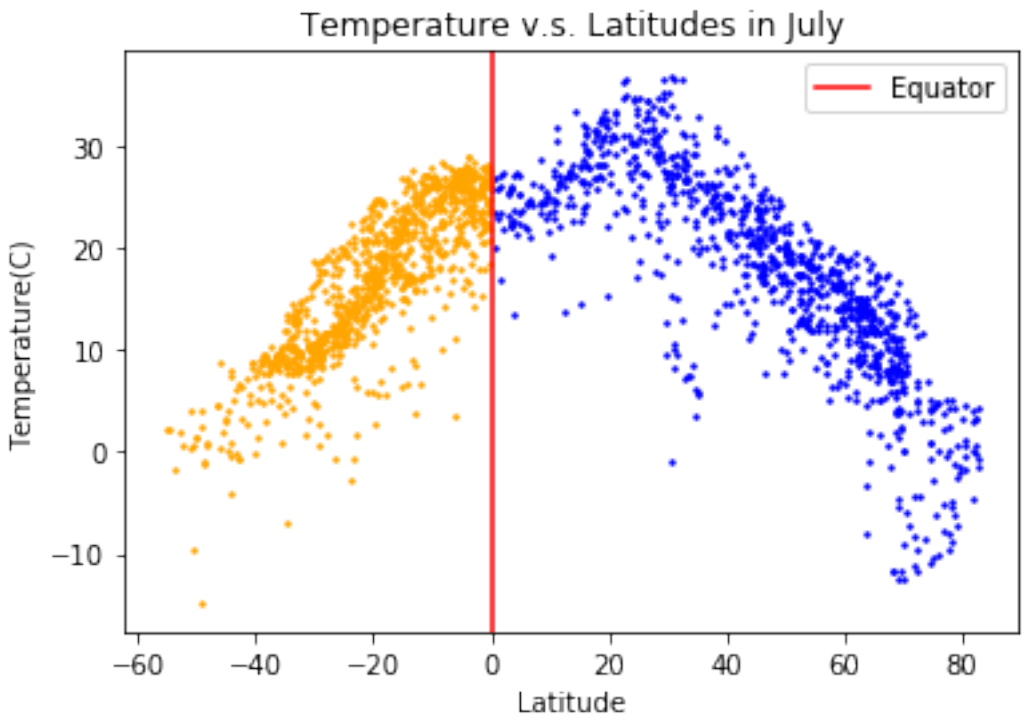
The correlation of both graphs got stronger because temperature is highly associated with season instead of latitude with temperature higher in the summer and lower in the winter, but latitude does decide what season a particular hemisphere is in. Since the northern and southern hemispheres always have opposite seasons, the correlation between latitude and temperature is weakened by the fact that both continents usually have different seasons. When we separate the northern hemisphere from the southern hemisphere, the correlation is significantly stronger because that half of the globe shares the same season.

0.1.8 1h)

I made two changes to make the graph more intuitively explanatory. First, I use different colors for the southern hemisphere and northern hemisphere to suggest the previous discovery. Even though the graph as a whole has a relatively weak correlation, if we separate the hemispheres a stronger correlation between temperature and latitude reveals. I strengthen this idea by drawing the equator between the southern hemisphere and the northern hemisphere, to further distinguish both hemispheres and help the reader recognize the correlation between temperature and latitude within each hemisphere.

[20]:
```
plt.scatter(df_north['latitude'],df_north['7'],s=2,color = 'blue')
plt.scatter(df_south['latitude'],df_south['7'],s=2,color = 'orange')
plt.xlabel('Latitude')
plt.ylabel('Temperature(C)')
plt.title('Temperature v.s. Latitudes in July')
plt.axvline(x=0, color='red',label='Equator')
plt.legend()
plt.show()
```

```
The history saving thread hit an unexpected error (OperationalError('attempt to
write a readonly database')).History will not be written to the database.
```



0.2 Problem 2

[3]:
```
import pandas as pd


df = pd.read_csv('shared/data/momiq.csv')
print(df.head())
```

```
   kid_score  mom_hs      mom_iq  mom_work  mom_age
0         65       1  121.117529         4       27
1         98       1   89.361882         4       25
2         85       1  115.443165         4       27
3         83       1   99.449639         3       25
4        115       1   92.745710         4       27
```

0.2.1 2a)

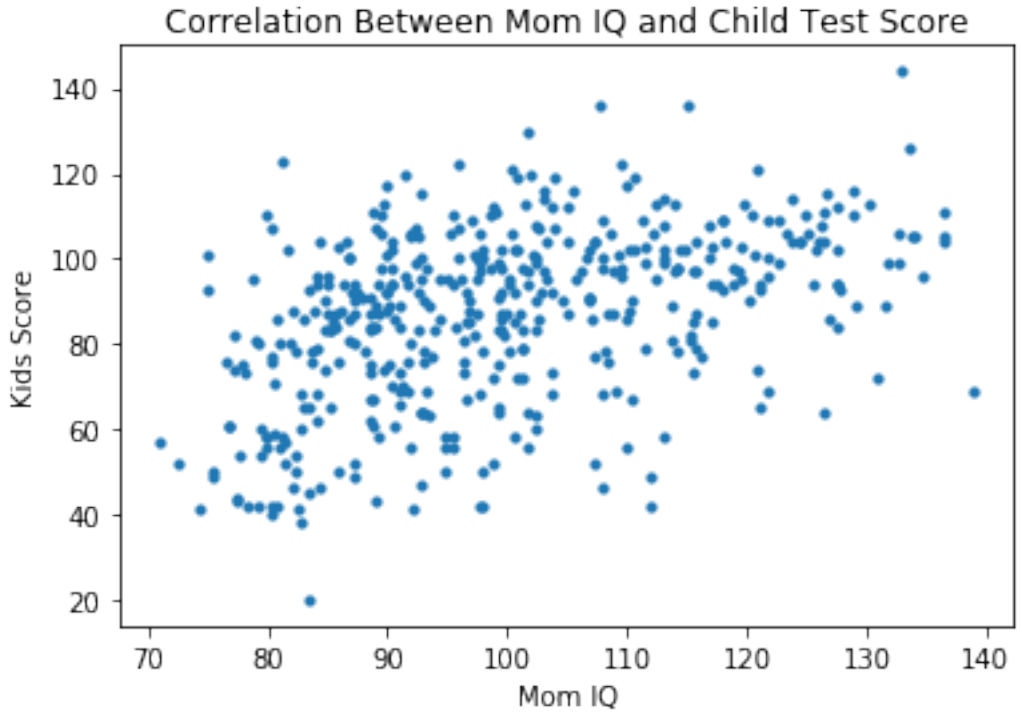
The formula for the slope of the regression is p * (STD of Y/STD of X)

0.2.2 2b)

I chose mother's IQ as explanatory variable X and Children's test score as response variable Y because IQ could be heritable from the parent, and it is reasonable to hypothesize that children with higher IQ will do better on the test.

[11]:
```
plt.scatter(df.mom_iq,df.kid_score, s=10)
plt.xlabel('Mom IQ')
plt.ylabel('Kids Score')
plt.title('Correlation Between Mom IQ and Child Test Score')
```

[11]:
```
Text(0.5, 1.0, 'Correlation Between Mom IQ and Child Test Score')
```



0.2.3 2c)

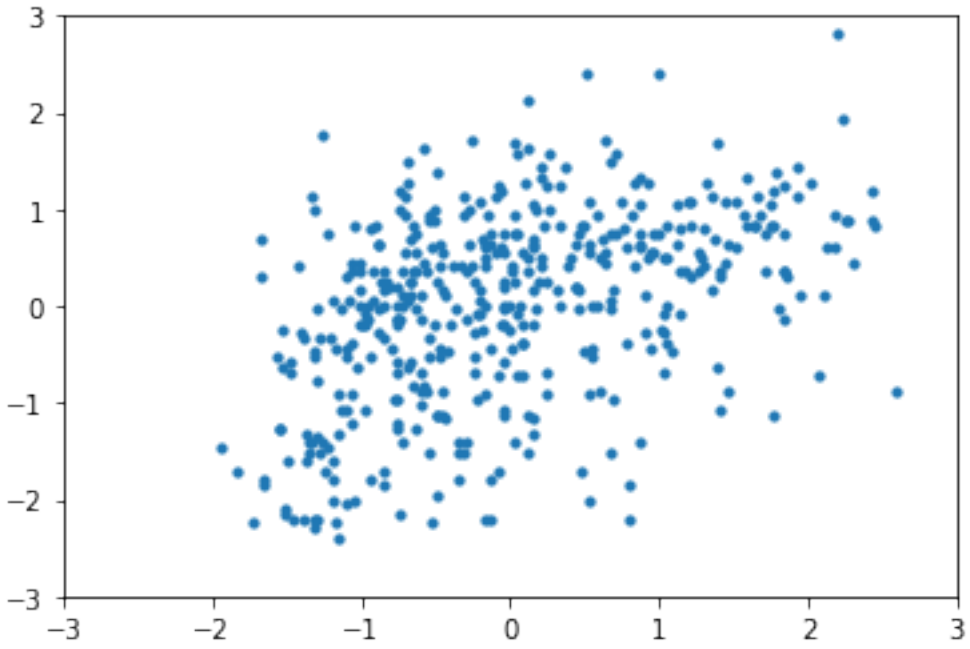
[5]:
```
import numpy as np
def standardize(df,column_name):
    std = np.std(df[column_name]) #calculating standard deviation
    mean = np.mean(df[column_name]) #calculating mean
    df_standardized = (df[column_name] - mean)/std #standardized column =␣
 ↪calculate Z-score for each value
    return df_standardized
```

0.2.4 2d)

Although the standardized version of the graph shifts slightly to the right compared to the earlier version, the two graphs remain very similar. The similarity between the earlier version and the standardized version suggests that the distribution of the two data follows a normal distribution, with the majority of data falling within 2 standard deviations from the mean.

[44]:
```
standardized_mom_iq = standardize(df,'mom_iq')
standardized_kid_score = standardize(df, 'kid_score')
plt.scatter(standardized_mom_iq,standardized_kid_score,s=10)
plt.xlim(-3,3)
plt.ylim(-3,3)
print(standardized_mom_iq)
```

```
0      1.409460
1     -0.710026
2      1.030732
3     -0.036733
4     -0.484177
         …
429   -1.009336
430   -0.467847
431   -0.343082
432   -0.209800
433   -0.583784
Name: mom_iq, Length: 434, dtype: float64
```



0.2.5 2e)

The slope 0.4483 suggests that for every one standard deviation increment in mom's IQ the child score on test would increase 0.4483 standard deviation.

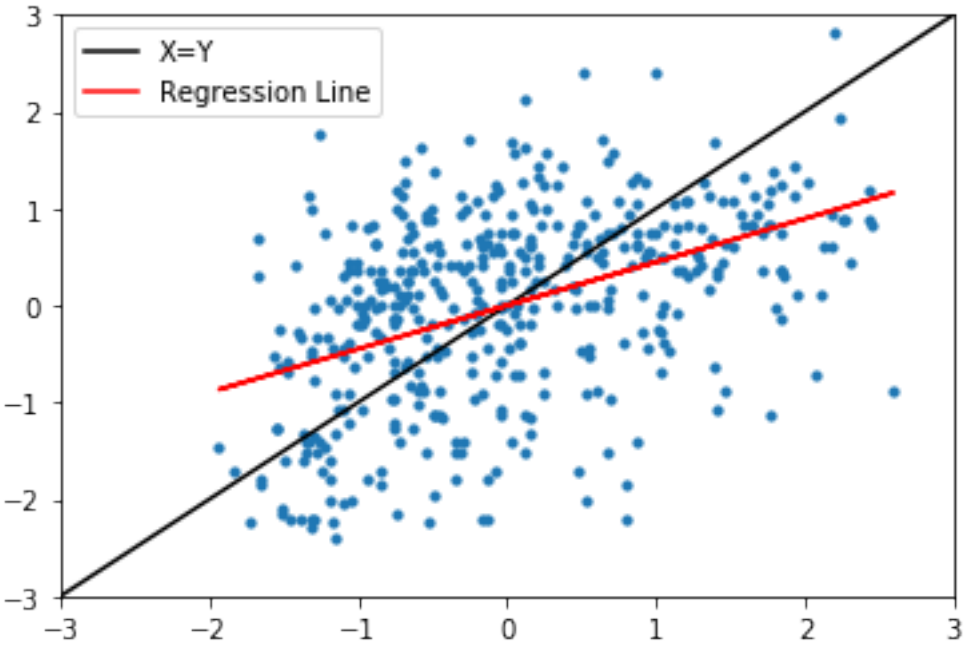
[49]:
```
import statsmodels.formula.api as smf
combined_df = pd.concat([standardized_mom_iq, standardized_kid_score], axis=1)
model = smf.ols(formula = 'standardized_mom_iq ~ standardized_kid_score',data =␣
↪combined_df)
result = model.fit()

print(result.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:     standardized_mom_iq   R-squared:                       0.201
Model:                             OLS   Adj. R-squared:                  0.199
Method:                  Least Squares   F-statistic:                     108.6
Date:                 Sun, 03 Nov 2024   Prob (F-statistic):           7.66e-23
Time:                         15:43:59   Log-Likelihood:                -567.14
No. Observations:                  434   AIC:                             1138.
Df Residuals:                      432   BIC:                             1146.
Df Model:                            1
Covariance Type:             nonrobust
==========================================================================================
                             coef    std err          t      P>|t|      [0.025
0.975]
------------------------------------------------------------------------------------------
Intercept              -2.992e-16      0.043  -6.96e-15      1.000      -0.085
0.085
standardized_kid_score     0.4483      0.043     10.423      0.000       0.364
0.533
==============================================================================
Omnibus:                       17.254   Durbin-Watson:                   1.748
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               17.687
Skew:                           0.465   Prob(JB):                     0.000144
Kurtosis:                       2.661   Cond. No.                         1.00
==============================================================================

Warnings:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```

0.2.6 2f)

[64]:
```
slope, intercept = np.polyfit(standardized_mom_iq, standardized_kid_score, 1) ␣
↪# 1 for linear fit
y_fit = slope * standardized_mom_iq + intercept

plt.scatter(standardized_mom_iq,standardized_kid_score,s=10)
```

```
plt.xlim(-3,3)
plt.ylim(-3,3)
x = np.linspace(-3, 3, 10)
y=x
plt.plot(x,y,color = 'black',label = 'X=Y')
plt.plot(standardized_mom_iq,y_fit, color = 'red', label = 'Regression Line')
plt.legend()
plt.show()
```



0.2.7 2g)

The regression line fits better. The slope of regression line is flatter compared to y=x. There are some data points that are pretty far from the y=x line, which causes a greater sum of squared residual, while the regression line are relatively closer to those data points.

0.3 Problem 3

[2]:
```
df=pd.read_csv('shared/data/bread_votes.csv')
print(df.head())
print('-----------------------------------------------------------------')
print('Mean of avg_recent_growth is', round(np.mean(df.avg_recent_growth),2))
print()
print('Standard deviation of avg_recent_growth is', round(np.std(df.
  ↪avg_recent_growth),2))
print()
```

```
print('There are ',len(df),'rows in the dataset')
```

```
   demvotes incumbent_party  repvotes  year  total_votes  \
0  24179347        democrat  21991292  1948     46170639
1  27375090        democrat  34075529  1952     61450619
2  26028028      republican  35579180  1956     61607208
3  34220984      republican  34108157  1960     68329141
4  43129040        democrat  27175754  1964     70304794

   incumbent_party_share  avg_recent_growth      dem_candidate  \
0               0.523695           2.945063    Harry S. Truman
1               0.445481           3.491052    Adlai Stevenson
2               0.577517           2.818698    Adlai Stevenson
3               0.499174           0.126799    John F. Kennedy
4               0.613458           4.391619  Lyndon B. Johnson

          rep_candidate                winner  incumbent_victory
0       Thomas E. Dewey       Harry S. Truman               True
1  Dwight D. Eisenhower  Dwight D. Eisenhower              False
2  Dwight D. Eisenhower  Dwight D. Eisenhower               True
3         Richard Nixon       John F. Kennedy              False
4       Barry Goldwater     Lyndon B. Johnson               True
----------------------------------------------------------------------
Mean of avg_recent_growth is 2.5

Standard deviation of avg_recent_growth is 1.36

There are  19 rows in the dataset
```

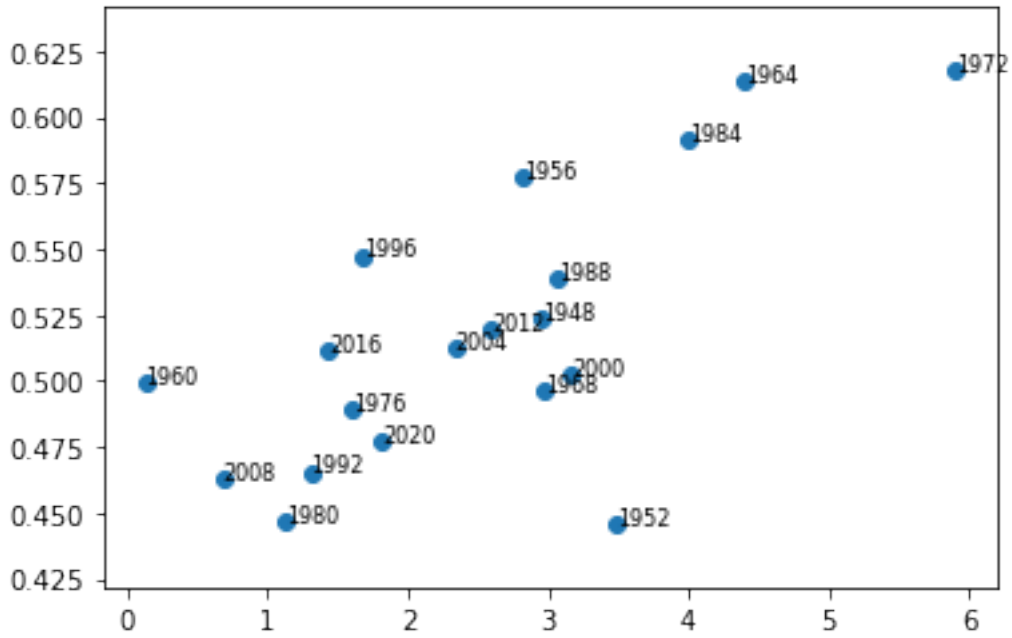
0.3.1 3c)

Average recent growth has a moderate to moderately strong positive correlation with the incumbent party share in the sample.

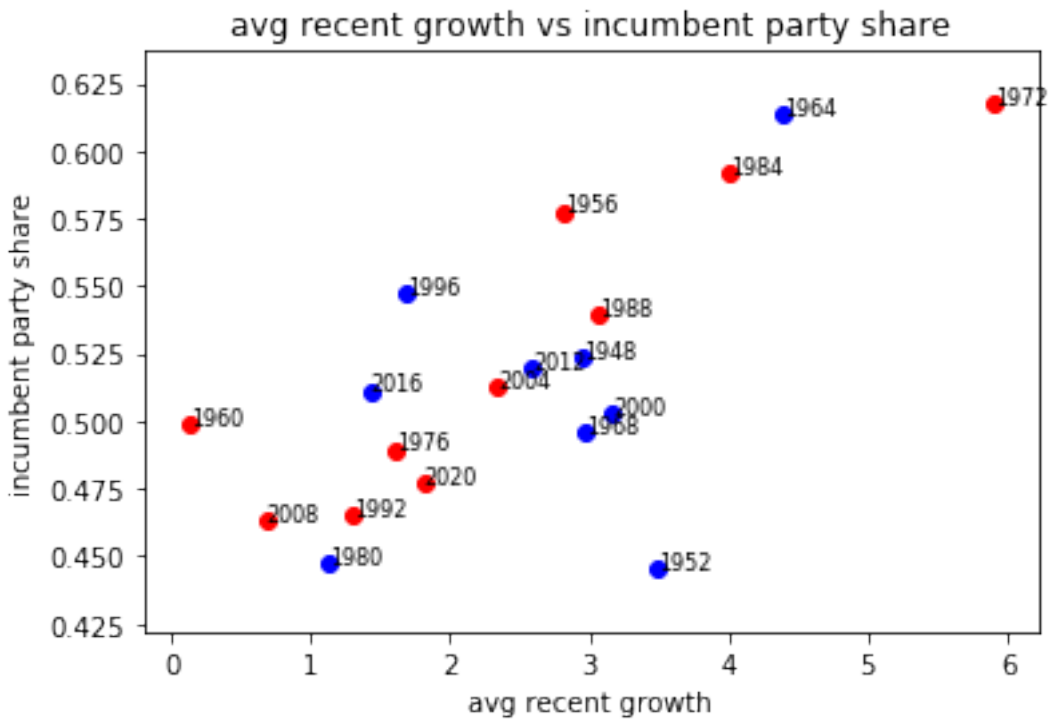
From the second graph, we can see that with a higher average recent growth, the Republicans tend to have a higher incumbent party share. This might be due to the Republican Party's laissez-faire attitude toward the economy, allowing individuals to pay fewer taxes, which could significantly boost residents' personal income if the Republican Party is still in place next term.

[27]:
```
#simple scatter plot with average recent growth as x-axis and incumbent party␣
 ↪share as y-axis
plt.scatter(df.avg_recent_growth,df.incumbent_party_share)
plt.xlabel("avg recent growth")
plt.ylabel("incumbent party share")
plt.title("avg recent growth vs incumbent party share")

#Iterate rows to get the year corresponding to each data point
```

```
for i, row in df.iterrows():#Generate an iterator that help me access␣
 ↪information each row
    plt.text(row['avg_recent_growth'],␣
 ↪row['incumbent_party_share'],row['year'],fontsize=8)
```



[28]:
```
plt.scatter(df.avg_recent_growth[df.incumbent_party=='democrat'],df.
 ↪incumbent_party_share[df.incumbent_party=='democrat'], color ='blue')
plt.scatter(df.avg_recent_growth[df.incumbent_party=='republican'],df.
 ↪incumbent_party_share[df.incumbent_party=='republican'], color ='red')
plt.xlabel("avg recent growth")
plt.ylabel("incumbent party share")
plt.title("avg recent growth vs incumbent party share")

#Iterate rows to get the year corresponding to each data point
for i, row in df.iterrows():#Generate an iterator that help me access␣
 ↪information each row
    plt.text(row['avg_recent_growth'],␣
 ↪row['incumbent_party_share'],row['year'],fontsize=8)
```



[87]:
```
correlation = df['avg_recent_growth'].corr(df['incumbent_party_share'])
print(round(correlation,3))
```

```
0.697
```

0.3.2 3d)

When the average recent growth is 0, the model's expected incumbent party share is 0.4526. The slope of the regression line is 0.0261, which means that for every 1 percent increment of average recent growth, the incumbent party shares increase by 0.0261.

[4]:
```
import statsmodels.formula.api as smf
model = smf.ols(formula = 'incumbent_party_share ~ avg_recent_growth',data = df)
result = model.fit()

print(result.summary())
```

```
                            OLS Regression Results
==============================================================================
=
Dep. Variable:     incumbent_party_share   R-squared:
0.486
Model:                               OLS   Adj. R-squared:
0.456
Method:                    Least Squares   F-statistic:
```

```
16.08
Date:                Tue, 05 Nov 2024   Prob (F-statistic):
0.000907
Time:                        09:51:36   Log-Likelihood:
35.970
No. Observations:                  19   AIC:
-67.94
Df Residuals:                      17   BIC:
-66.05
Df Model:                           1
Covariance Type:            nonrobust
================================================================================
=====
                     coef    std err          t      P>|t|      [0.025
0.975]
--------------------------------------------------------------------------------
-----
Intercept              0.4526      0.019     24.456      0.000       0.414
0.492
avg_recent_growth      0.0261      0.007      4.010      0.001       0.012
0.040
==============================================================================
Omnibus:                        3.434   Durbin-Watson:                   2.486
Prob(Omnibus):                  0.180   Jarque-Bera (JB):                1.621
Skew:                          -0.646   Prob(JB):                        0.445
Kurtosis:                       3.616   Cond. No.                         6.54
==============================================================================

Warnings:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```

0.3.3 3e)

The predicted incumbent party's popular vote share in 2024 would be rounded to 51.71%.

[13]:
```
x_new = pd.DataFrame({'avg_recent_growth':[2.47]})
y_new = result.predict(x_new).values[0]
y_new
```

[13]:
```
0.5171393624426903
```

0.3.4 3f)

[7]:
```
n = len(df)

residual_array = np.array(result.resid)
```

```
s = np.sqrt((np.square(residual_array)).sum()/(n-2))

x_new = 2.47

x_mean = np.mean(df.avg_recent_growth)

squared_deviation = 0
sum_of_square_deviation = 0
for i in range(n):#Square each differences and sum it up
    squared_deviation = np.square(df["avg_recent_growth"][i] - x_mean)
    sum_of_square_deviation += squared_deviation


SE_pred = s*(np.sqrt(1+(1/n)+ (np.square(x_new-x_mean))/
 ↪sum_of_square_deviation))

#check for potential error in each parts
print('Standard error =', SE_pred)
print("s:", s)
print("n:", n)
print("x_mean:", x_mean)
print("x_new:", x_new)
print("sum_of_square_deviation:", sum_of_square_deviation)
print("Expression inside sqrt:", 1 + (1/n) + (np.square(x_new - x_mean) /␣
 ↪sum_of_square_deviation))
```

```
Standard error = 0.03952570770428917
s: 0.038524492816135424
n: 19
x_mean: 2.4977206998443435
x_new: 2.47
sum_of_square_deviation: 35.01265689840284
Expression inside sqrt: 1.0526535263591883
```

[8]:
```
upper_bound = round(0.5171393624426903+2*SE_pred,2)
lower_bound = round(0.5171393624426903-2*SE_pred,2)
[lower_bound,upper_bound]
```

[8]:
```
[0.44, 0.6]
```

[ ]:
```

```